

# LangGraph Mastery Guide — Phase 5: Multi-Agent & Tools

**Tech stack used throughout this guide:** Node.js + TypeScript, @langchain/langgraph, @langchain/openai (for the LLM calls), @langchain/core, and zod (for schemas). Every code sample is written to run in a real TypeScript project — not pseudocode.

## How to Use This Guide

This is Phase 5 of your LangGraph roadmap, covering three units:

| Unit | Title                       | What it teaches you                                                                                                                              |
|------|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 9    | Subgraphs                   | How to nest one graph inside another to build modular, reusable workflows                                                                        |
| 10   | Multi-Agent Architectures   | How to coordinate multiple specialized “agents” (Supervisor, Network, Hierarchical Teams, Swarm)                                                 |
| 11   | Tools & Prebuilt Components | How to give agents the ability to call functions (tools), and how to use LangGraph’s ready-made agent builder instead of hand-rolling everything |

This guide assumes you’ve already been through Phases 1–4 of the roadmap (Foundation, Control Flow, Persistence & Memory, Human-in-the-Loop). That means you should already be comfortable with: what a StateGraph is, how nodes and edges work, what a reducer does, how addConditionalEdges and the Command object work, and what a checkpointer is. If any of those terms feel shaky, it’s worth a quick look back at the earlier phases before diving in — this guide builds directly on top of them.

**Every topic below follows the same four-part structure:** 1. **Concept and Theory** — plain-language explanation, no assumed prior knowledge of the specific feature. 2. **Practical Lab** — step-by-step, hands-on, explaining *why* each line of code exists. 3. **Knowledge Check** — summary, takeaways, questions, and a small exercise. 4. At the end of the whole phase, a **Final Project** ties Units 9, 10, and 11 together into one real system.

## Before You Start: Project Setup

You’ll reuse this same project folder for every lab in this guide. Let’s set it up once, properly, so you never have to think about it again.

## Why we're doing this first

Every lab below assumes a working TypeScript + Node.js project with the right packages installed and your OpenAI API key configured. If this step is skipped, every single code sample later will fail with confusing module errors — so we do it carefully, once, up front.

## Step-by-step setup

### 1. Create a project folder and initialize it.

```
mkdir langgraph-phase5
cd langgraph-phase5
npm init -y
```

*What this does:* `npm init -y` creates a `package.json` file with default values. This file is Node's "manifest" — it tracks what packages your project depends on, so anyone (including future you) can reinstall everything with one command.

*Expected result:* A new `package.json` file appears in the folder. Open it — you'll see fields like "name", "version", "main". That's normal; we'll edit a couple of these in a moment.

### 2. Install TypeScript and the LangGraph packages.

```
npm install typescript tsx @types/node --save-dev
npm install @langchain/langgraph @langchain/core @langchain/openai
zod
```

*What each package is for:* - `typescript` — the TypeScript compiler itself. - `tsx` — lets us `run .ts` files directly with `npx tsx file.ts`, without a separate compile step. This is purely a convenience for learning; in production you'd typically compile first. - `@types/node` — type definitions for Node's built-in APIs (like `process.env`), so TypeScript doesn't complain when we read environment variables. - `@langchain/langgraph` — the core LangGraph library: `StateGraph`, `Annotation`, `Command`, `ToolNode`, `createReactAgent`, etc. - `@langchain/core` — shared building blocks used across the LangChain ecosystem, including message types (`HumanMessage`, `AIMessage`, `ToolMessage`) and the `tool()` helper for defining tools. - `@langchain/openai` — the OpenAI chat model wrapper (`ChatOpenAI`), which is how LangGraph nodes will actually talk to an LLM. - `zod` — a schema-definition library. We use it to describe the *shape* of our state and our tool inputs in a way both TypeScript and LangGraph understand.

*Expected result:* A `node_modules` folder appears, and `package.json` gains a "dependencies" and "devDependencies" section listing these packages.

*Common beginner mistake:* Forgetting `--save-dev` for `typescript/tsx/@types/node` doesn't break anything functionally, but it's good hygiene — those are *development* tools, not something your running application needs in production. Don't worry if you forget; it won't cause lab failures.

### 3. Add a `tsconfig.json`.

Create a file named `tsconfig.json` in the project root with this content:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "esModuleInterop": true,
    "strict": true,
    "skipLibCheck": true,
  }
}
```

```

    "outDir": "dist"
  },
  "include": ["src/**/*.ts"]
}

```

*Why this matters:* "module": "NodeNext" and "moduleResolution": "NodeNext" tell TypeScript to use Node's modern module system, which is what @langchain/langgraph expects. If you use an older module setting, you'll get cryptic "cannot find module" errors even though the package is installed correctly.

#### 4. Set your OpenAI API key.

Create a .env file in the project root:

```
OPENAI_API_KEY=sk-your-key-here
```

Then install dotenv so Node can read it:

```
npm install dotenv
```

At the top of every lab file in this guide, you'll see:

```
import "dotenv/config";
```

*What this does:* loads the .env file's contents into process.env before anything else in the file runs, so new ChatOpenAI({...}) can find your API key automatically (the ChatOpenAI class reads process.env.OPENAI\_API\_KEY by default — you don't need to pass it explicitly).

*Common beginner mistake:* Putting import "dotenv/config" *after* you've already created a ChatOpenAI instance. Import order matters here — it must be the very first import in the file.

#### 5. Create a src/ folder.

```
mkdir src
```

All lab files in this guide will live in src/, e.g. src/lab9-1.ts. You'll run them with:

```
npx tsx src/lab9-1.ts
```

#### 6. Verify the setup works.

Create src/sanity-check.ts:

```

import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0
});

const response = await model.invoke("Say 'setup works' and nothing
else.");
console.log(response.content);

```

Run it:

```
npx tsx src/sanity-check.ts
```

*Expected result:* The terminal prints setup works (or very close to it). If you see this, your environment is correctly configured for every lab in this guide.

*If it fails:* The two most common causes are (a) a missing or invalid OPENAI\_API\_KEY in .env, or (b) the .env file not being in the same folder you're running the command from. Double check both before moving

on.

---

## Chapter 9 — Subgraphs

### 9.1 Concept and Theory

#### What is a subgraph?

A **subgraph** is simply a *compiled LangGraph graph that gets used as a single node inside another graph*. That's the whole idea. You already know how to build a graph with `StateGraph` — define a state schema, add nodes, wire them with edges, compile it. A subgraph takes that exact compiled object and plugs it into a *bigger* graph, where, from the outside, it behaves like any other node: it receives state, does some work (which might involve many internal steps), and returns updated state.

#### Why does this exist?

Think about how you organize a large codebase. You don't write one giant 2,000-line function — you break it into smaller functions, each with a clear job, and you call them from a higher-level function. Subgraphs let you do the exact same thing for *agent workflows*.

Without subgraphs, a complex agent system becomes one enormous flat graph with dozens of nodes and a tangle of conditional edges, which is hard to read, hard to test, and hard to reuse. With subgraphs, you can build a self-contained “research” workflow once — with its own internal nodes for searching, summarizing, and fact-checking — and then drop that entire workflow into any parent graph that needs research capability, the same way you'd import and call a function.

#### When do you use a subgraph?

Reach for a subgraph when:

- You have a *cohesive chunk of logic* that has its own multi-step internal process (e.g., “do research,” “write and revise a draft,” “validate compliance”) and that chunk could conceptually stand alone.
- You want to **reuse** the same workflow inside multiple different parent graphs (e.g., a “research” subgraph used both inside a “blog writer” graph and inside a “customer support” graph).
- You're building a **multi-agent system** (which you'll do in Chapter 10) — each “team” of agents is very often implemented as a subgraph, and a supervisor graph coordinates between several team-subgraphs.
- Your single flat graph has grown large enough that you're struggling to reason about it, and you can identify a natural seam to split it along.

#### How it works under the hood

When you call `.compile()` on a `StateGraph`, you get back a `CompiledStateGraph` object. This compiled object is actually a `Runnable` — the same base interface that every `LangChain` component implements (chat models, chains, tools, etc.). Because nodes in a parent graph are just functions (or anything `Runnable-compatible`) that take state and return a partial state update, and because a compiled graph satisfies that same shape, you can literally do:

```
parentGraph.addNode("research", compiledResearchSubgraph);
```

and LangGraph treats the entire subgraph as one node. When the parent graph “calls” that node, it actually runs the *entire* internal subgraph (potentially many of its own nodes and edges) from its START to its END, and only then returns control to the parent.

## The state-mapping problem

Here’s the part that trips people up: **the parent graph and the subgraph each have their own state schema**, and those schemas don’t have to match.

There are two situations:

**Situation A — shared keys (the easy case).** If the subgraph’s state schema shares one or more key names with the parent’s state schema (e.g., both have a `messages` key), LangGraph can automatically pass those shared keys into the subgraph and merge the subgraph’s updates back into the parent state. You don’t have to write any translation code — just add the compiled subgraph as a node, and the matching keys flow through automatically.

**Situation B — no shared keys, or you need a transformation.** If the parent’s state shape is structurally different from what the subgraph expects (e.g., parent has `customerQuestion: string` but the subgraph expects `messages: BaseMessage[]`), automatic merging can’t help you, because LangGraph has no way to know `customerQuestion` should become the first message in `messages`. In this case, you don’t add the compiled subgraph directly as a node. Instead, you wrap it in a regular function-node that: 1. Reads the relevant fields from parent state. 2. Constructs the subgraph’s expected input shape. 3. Calls `.invoke()` on the compiled subgraph with that input. 4. Takes the subgraph’s output and translates it back into a partial *parent* state update.

This wrapper function is what gets added to the parent graph with `addNode()` — the subgraph is invoked *inside* it, not passed in directly.

## Streaming and checkpointing across subgraph boundaries

Two important details about subgraphs interacting with features you’ve already learned:

- **Streaming:** When you stream a parent graph that contains a subgraph, by default you only see the parent’s node-level updates (the subgraph as a whole “fires” once). If you want to see the subgraph’s *internal* steps streamed too, you need to pass `subgraphs: true` as an option to `.stream()`. This is incredibly useful for debugging — without it, a slow subgraph looks like one long silent pause from the outside.
- **Checkpointing:** If the parent graph is compiled with a checkpointer (e.g., `MemorySaver`), and the subgraph is compiled *without* its own checkpointer, the subgraph automatically inherits and uses the parent’s checkpointer for its own internal state — so time travel and resumability extend into the subgraph for free. If the subgraph is compiled with its *own* checkpointer, that checkpointer is used instead, and the subgraph effectively gets independently-resumable internal state. For most use cases, letting the subgraph inherit the parent’s checkpointer (i.e., not giving the subgraph its own) is the simpler and more predictable choice.

## A useful mental model

Think of a subgraph the way you'd think of a microservice. From the outside, callers only care about its input and output contract (the state keys it consumes and produces). What happens inside — how many internal steps, what intermediate state it tracks — is private implementation detail that the parent doesn't need to know about. That encapsulation is the entire value of subgraphs.

---

## 9.2 Practical Lab

We're going to build a "research" subgraph and embed it into two different parent graphs — first one with overlapping state keys (Situation A), then one with non-overlapping state keys requiring an explicit wrapper (Situation B). Then we'll look at streaming through subgraph boundaries and reusing one subgraph in two parents.

### Lab 9.1 — Build a standalone "research" subgraph and embed it in a larger graph

**Goal:** Create a self-contained 2-node subgraph (search → summarize) and plug it into a parent graph as a single node, using the "shared keys" automatic-merge approach.

#### Step 1 — Define the subgraph's own state.

Create `src/lab9-1.ts`:

```
import "dotenv/config";
import { StateGraph, Annotation, START, END } from
"@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";

// The subgraph's own, self-contained state.
const ResearchState = Annotation.Root({
  topic: Annotation<string>,
  searchResults: Annotation<string>,
  summary: Annotation<string>,
});
```

*Why we do this:* The subgraph is meant to be reusable and self-contained, so it gets its *own* state schema, independent of whatever parent graph might eventually use it. `topic` is the input it needs, `searchResults` and `summary` are intermediate/output fields it produces internally.

#### Step 2 — Build the subgraph's two nodes.

```
const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0
});

async function searchNode(state: typeof ResearchState.State) {
  // In a real system this would call a search API (e.g., Tavily).
  // We simulate it here with an LLM call so the lab needs no extra
  API keys.
  const response = await model.invoke(
    `Pretend you searched the web for "${state.topic}". Write 3
    short bullet-point facts you "found."`
  );
  return { searchResults: response.content as string };
}

async function summarizeNode(state: typeof ResearchState.State) {
  const response = await model.invoke(
    `Summarize these research findings into one concise
    paragraph:\n\n${state.searchResults}`
```

```

    );
    return { summary: response.content as string };
}

```

*Why we do this:* searchNode reads topic from state and writes searchResults. summarizeNode reads searchResults and writes summary. Each node only touches the part of state it cares about — this is normal LangGraph node design, nothing subgraph-specific yet.

*Note:* We simulate search with an LLM call to keep this lab dependency-free. In a real project, you'd replace this with an actual search tool call (you'll learn the tool-calling pattern properly in Chapter 11).

### Step 3 — Wire and compile the subgraph.

```

const researchSubgraph = new StateGraph(ResearchState)
  .addNode("search", searchNode)
  .addNode("summarize", summarizeNode)
  .addEdge(START, "search")
  .addEdge("search", "summarize")
  .addEdge("summarize", END)
  .compile();

```

*What this does:* This is a complete, independently runnable graph at this point. You could call researchSubgraph.invoke({ topic: "..." }) right now and get back a full ResearchState object. We're about to nest it inside something bigger instead.

*Verify it works standalone first* (good practice before nesting anything):

```

const standaloneResult = await researchSubgraph.invoke({ topic: "the
history of the printing press" });
console.log("Standalone subgraph result:",
standaloneResult.summary);

```

Run with `npm run tsx src/lab9-1.ts` — you should see a one-paragraph summary printed. This confirms the subgraph works on its own *before* we complicate things by nesting it.

### Step 4 — Define a parent graph with an overlapping key.

```

const ParentState = Annotation.Root({
  topic: Annotation<string>, // shared key name with
ResearchState
  summary: Annotation<string>, // shared key name with
ResearchState
  finalReport: Annotation<string>,
});

async function writeReportNode(state: typeof ParentState.State) {
  const response = await model.invoke(
    `Turn this research summary into a short, polished report titled
about "${state.topic}":\n\n${state.summary}`
  );
  return { finalReport: response.content as string };
}

```

*Why the shared keys matter:* ParentState has topic and summary — the exact same field names as ResearchState. This is what enables automatic merging in Step 5.

### Step 5 — Add the compiled subgraph directly as a parent node.

```

const parentGraph = new StateGraph(ParentState)
  .addNode("research", researchSubgraph) // <-- the compiled
subgraph IS the node

```

```

        .addNode("writeReport", writeReportNode)
        .addEdge(START, "research")
        .addEdge("research", "writeReport")
        .addEdge("writeReport", END)
        .compile();

const result = await parentGraph.invoke({ topic: "the history of the
printing press" });
console.log("\n--- FINAL REPORT ---\n", result.finalReport);

```

*What's happening:* When the parent graph reaches the "research" node, LangGraph sees that the node is itself a compiled graph. It takes the current parent state, extracts the keys that match the subgraph's schema (topic), runs the *entire* subgraph from its START to its END internally, and then merges the subgraph's output keys (topic, searchResults — wait, note searchResults *isn't* in ParentState, more on that below — and summary) back into parent state wherever the key names match.

*Important detail:* searchResults exists in the subgraph's state but not in the parent's. That's fine — LangGraph only merges back keys that exist in *both* schemas. searchResults simply doesn't make it into the parent state, and that's correct behavior: the parent never asked for it.

**Expected result:** Running the file prints a polished final report to the terminal, having gone through search → summarize (inside the subgraph) → write report (in the parent), with zero manual plumbing between the subgraph and the parent.

**How to verify it worked correctly:** Add a console.log(result) at the end and confirm the returned object has topic, summary, and finalReport populated — but *not* searchResults (proving only matching keys merged back).

**Common beginner mistakes:** - Forgetting that key *names* must match exactly (case-sensitive) for automatic merging to work — Topic and topic are different keys and will not merge. - Assuming all subgraph fields flow into the parent. Only fields that exist in *both* schemas merge; everything else stays internal to the subgraph.

---

## Lab 9.2 — Map a subset of parent state into a subgraph's own schema

**Goal:** Handle Situation B — a parent graph whose state shape does *not* overlap with the subgraph's expected input — by wrapping the subgraph call in a translation function.

### Step 1 — Imagine a parent with an incompatible shape.

```

const SupportState = Annotation.Root({
  customerQuestion: Annotation<string>, // no "topic" field here
  researchSummary: Annotation<string>, // no "summary" field here
  customerReply: Annotation<string>,
});

```

*Why this is Situation B:* This parent has no field called topic and no field called summary. If you tried addNode("research", researchSubgraph) directly here, nothing would merge — customerQuestion and topic are different names, so the subgraph would receive topic: undefined and silently fail to do useful work.

### Step 2 — Write a translation wrapper node instead of adding the subgraph directly.



```

    async function researchWrapperNode(state: typeof SupportState.State)
    {
        // Step A: translate parent state -> subgraph input shape
        const subgraphInput = { topic: state.customerQuestion };

        // Step B: invoke the subgraph explicitly
        const subgraphResult = await
researchSubgraph.invoke(subgraphInput);

        // Step C: translate subgraph output -> partial parent state
update
        return { researchSummary: subgraphResult.summary };
    }

```

*Why we do it this way:* This three-step pattern (translate in → invoke → translate out) is the general-purpose solution for *any* schema mismatch, no matter how different the shapes are. It also keeps the translation logic visible and testable in one place, rather than relying on “magic” merging that wouldn’t work here anyway.

### Step 3 — Wire the parent graph using the wrapper, not the raw subgraph.

```

    async function replyNode(state: typeof SupportState.State) {
        const response = await model.invoke(
            `A customer asked: "${state.customerQuestion}". Using this
research: "${state.researchSummary}", write a brief, friendly
reply.`
        );
        return { customerReply: response.content as string };
    }

    const supportGraph = new StateGraph(SupportState)
        .addNode("research", researchWrapperNode) // <-- wrapper node,
not the raw subgraph
        .addNode("reply", replyNode)
        .addEdge(START, "research")
        .addEdge("research", "reply")
        .addEdge("reply", END)
        .compile();

    const supportResult = await supportGraph.invoke({
        customerQuestion: "How was paper made before the printing press
existed?",
    });
    console.log("\n--- CUSTOMER REPLY ---\n",
supportResult.customerReply);

```

**Expected result:** A friendly customer-support-style reply is printed, even though SupportState and ResearchState share *zero* field names. The wrapper node did all the translation work.

**Verification step:** Temporarily remove the translation line (`const subgraphInput = { topic: state.customerQuestion }`) and instead pass `{}` directly to `researchSubgraph.invoke()`. Run it again — you should see the subgraph either error or produce a nonsensical research summary about an empty topic, which demonstrates exactly why the translation step is necessary.

**Common beginner mistake:** Trying to force automatic merging to work by renaming parent fields to match the subgraph (e.g., renaming `customerQuestion` to `topic` everywhere). This works for a quick demo but breaks the *point* of subgraphs being independently designed, reusable components — you shouldn’t have to redesign your parent’s state just to satisfy one subgraph’s naming. The wrapper pattern is the scalable solution.

---

## Lab 9.3 — Stream output through a graph that contains a nested subgraph

**Goal:** See the difference between default streaming (subgraph is a black box) and `subgraphs: true` streaming (you see inside it too).

### Step 1 — Stream the Lab 9.1 parent graph normally.

```
console.log("--- Default streaming (subgraph hidden) ---");
for await (const chunk of await parentGraph.stream(
  { topic: "the invention of the wheel" },
  { streamMode: "updates" }
)) {
  console.log(chunk);
}
```

*Expected result:* You'll see exactly two updates printed — one for the "research" node (the entire subgraph collapsed into a single update) and one for "writeReport". You will *not* see the subgraph's internal search and summarize steps individually.

### Step 2 — Re-run with `subgraphs: true`.

```
console.log("\n--- Streaming with subgraphs: true (subgraph
internals visible) ---");
for await (const chunk of await parentGraph.stream(
  { topic: "the invention of the wheel" },
  { streamMode: "updates", subgraphs: true }
)) {
  console.log(chunk);
}
```

*What changes:* Each streamed chunk is now a tuple of [namespace, update]. The namespace tells you *where* the update came from — for top-level parent nodes it'll be an empty path, but for nodes living inside the subgraph, the namespace will show something like the subgraph's node path, letting you see search and summarize firing individually, nested inside the "research" step.

**Why this matters in practice:** Without `subgraphs: true`, a slow or buggy subgraph just looks like a long, silent pause in your stream — you have no visibility into *which* internal step is slow or failing. Turning this on during development (and turning it back off for a cleaner production UI, if you don't want to expose internal steps to end users) is a very common debugging technique.

**Verification step:** Count the chunks in each run. The first loop should print exactly 2 chunks (research, writeReport). The second loop should print more than 2 chunks, because the subgraph's internal nodes now show up individually.

---

## Lab 9.4 — Reuse the same subgraph inside two different parent graphs

**Goal:** Prove the core value proposition of subgraphs — write once, use in multiple unrelated parents.

### Step 1 — Reuse `researchSubgraph` (from Lab 9.1) in a brand new, unrelated parent.

```
const BlogState = Annotation.Root({
  topic: Annotation<string>,
  summary: Annotation<string>,
  blogPost: Annotation<string>,
});
```

```

    async function blogWriterNode(state: typeof BlogState.State) {
      const response = await model.invoke(
        `Write a punchy 3-paragraph blog post about "${state.topic}"
        using this research:\n\n${state.summary}`
      );
      return { blogPost: response.content as string };
    }

    const blogGraph = new StateGraph(BlogState)
      .addNode("research", researchSubgraph) // SAME compiled subgraph
      object as Lab 9.1
      .addNode("blogWriter", blogWriterNode)
      .addEdge(START, "research")
      .addEdge("research", "blogWriter")
      .addEdge("blogWriter", END)
      .compile();

    const blogResult = await blogGraph.invoke({ topic: "why the printing
    press changed the world" });
    console.log("\n--- BLOG POST ---\n", blogResult.blogPost);

```

*What to notice:* We did not redefine, recompile, or copy-paste any part of `researchSubgraph`. The exact same compiled object that powered the report-writing parent graph in Lab 9.1 now powers a completely different blog-writing parent graph here, because `BlogState` also happens to share the `topic/summary` key names. This is precisely the “write a function once, call it from many places” benefit subgraphs are designed to give you.

**Verification step:** Add a `console.log("Same subgraph object?", researchSubgraph === researchSubgraph)` — trivially true, but the point is to confirm in your own code that you imported/referenced the *same* variable in both graphs rather than rebuilding it. In a multi-file project, this would mean exporting `researchSubgraph` from one module and importing it into two different parent-graph files.

**Common beginner mistake:** Recompiling the same subgraph logic separately for each parent “just to be safe.” This isn’t wrong, but it defeats the purpose — if you ever need to fix a bug in the research logic, you’d now have to fix it in two places instead of one. Reusing the *compiled* object is both more efficient and easier to maintain.

## 9.3 Knowledge Check — Subgraphs

### Summary

A subgraph is a compiled `StateGraph` used as a single node inside a bigger, parent `StateGraph`. This lets you build modular, testable, reusable chunks of agent logic instead of one giant flat graph. If the parent and subgraph state schemas share key names, `LangGraph` merges state automatically. If they don’t, you wrap the subgraph invocation in a regular node function that translates state in and out manually. Subgraphs inherit the parent’s checkpointers by default (unless given their own), and their internal steps are hidden from streaming output unless you pass `subgraphs: true`.

### Key Takeaways

- A subgraph is just `.compile()`’d output of a `StateGraph`, used as a node value instead of a plain function.
- **Shared key names → automatic merging.** No shared keys → write a wrapper node that translates state manually.

- Fields that exist only inside the subgraph’s schema never leak into the parent’s state.
- subgraphs: `true` on `.stream()` reveals internal subgraph steps; omitting it collapses the whole subgraph into one update.
- The same compiled subgraph object can be reused, unmodified, inside any number of different parent graphs.
- Subgraphs are the standard building block for multi-agent “teams,” which you’ll use heavily in Chapter 10.

## Practice Questions

1. If a subgraph’s state has a field called `draft` and the parent’s state has no field called `draft`, what happens to that field when the subgraph runs inside the parent?
2. Why can’t automatic state merging work when the parent has `customerQuestion: string` and the subgraph expects `topic: string`?
3. What’s the difference in what you see when streaming a parent graph with `subgraphs: true` versus without it?
4. If you compile a subgraph with its own `MemorySaver`, instead of letting it inherit the parent’s `checkpoint`, what changes about how its internal state is tracked?

## Exercises

1. Take the `researchSubgraph` from Lab 9.1 and add a third internal node, `factCheck`, that runs after `summarize` and revises the summary if it spots an obviously made-up “fact.” Confirm the parent graphs from Labs 9.1 and 9.4 still work unmodified.
2. Build a second, totally different subgraph (e.g., a 2-node “translate then proofread” subgraph) and embed it as a *second* node inside the `parentGraph` from Lab 9.1, so the flow becomes: `research` → `write report` → `translate-and-proofread`.

## Mini Project

Build a “**content pipeline**” **parent graph** with three subgraph nodes in sequence: `research` (from Lab 9.1) → `draft` (a subgraph that writes a first draft, then has an internal “self-critique” loop using a conditional edge that loops back to drafting up to 2 times) → `polish` (a subgraph that does tone adjustment and proofreading). Confirm the whole pipeline runs end-to-end with one `.invoke()` call from outside, and that streaming with `subgraphs: true` shows you the self-critique loop firing internally inside the `draft` subgraph.

---

# Chapter 10 — Multi-Agent Architectures

## 10.1 Concept and Theory

### What is a “multi-agent” system, really?

So far in this roadmap, “the agent” has meant one LLM-powered loop making decisions inside a graph. A **multi-agent system** is simply a graph where *multiple, separately-prompted LLM nodes* each play a specialized role, and the graph’s job is to route work between them correctly.

Think of it like a company. You don't hire one person to be the accountant, the lawyer, the engineer, and the customer service rep all at once and hope they're equally good at everything. You hire specialists, each with a narrow, well-defined job, and you build a process for routing work to the right specialist and combining their outputs. Multi-agent architectures are that same idea, applied to LLM-powered nodes in a graph.

## Why split into multiple agents instead of one big agent?

A single agent with a giant system prompt trying to handle “answer math questions, write prose, search the web, and manage a calendar” tends to perform worse at *all four* tasks than four separate, narrowly-prompted agents would at their one task each. This happens for a few concrete reasons:

- **Prompt focus.** A system prompt crammed with instructions for four different jobs dilutes the model's attention; a focused prompt for one job lets the model use its full context budget on that job.
- **Tool overload.** If one agent has access to 20 different tools (some for math, some for writing, some for calendars), it more often picks the wrong tool. Specialist agents can each have a small, relevant tool set.
- **Debuggability.** When something goes wrong, it's much easier to tell “the math agent gave a wrong answer” than to untangle which part of one mega-prompt failed.
- **Reusability.** A well-built “writer” agent can be reused across many different systems, the same way a subgraph can.

## The four architectures

LangGraph doesn't force you into one specific multi-agent pattern — it gives you the primitives (Command, conditional edges, subgraphs) to build whichever shape fits your problem. There are four common shapes worth knowing by name:

**1. Supervisor.** One central “supervisor” node acts as a router/dispatcher. It looks at the current state (e.g., the user's question) and decides which *one* worker agent should handle it next. The chosen worker does its job and then routes *back* to the supervisor, which either sends the task to another worker or finishes. Control always flows through the supervisor — workers never talk to each other directly.

*Analogy:* A receptionist at a hospital. You explain your problem once; the receptionist decides “you need to see Dr. Smith (cardiology),” you go see Dr. Smith, and if cardiology decides you also need a blood test, you don't walk straight to the lab yourself — you go back through the front desk, which routes you again.

*When to use it:* This is the most common, most predictable multi-agent pattern, and the right default choice for most systems. Use it whenever there's a clear “front door” decision (which specialist handles this?) and workers don't need to coordinate with each other directly.

**2. Network.** Every agent can potentially hand off directly to every other agent — there's no single central dispatcher. Each agent, after doing its work, decides for itself which agent (if any) should go next.

*Analogy:* A team of co-workers in an open office who can just turn to whichever colleague is relevant and say “can you take this from here?” without going through a manager first.

*When to use it:* Useful when the “next step” genuinely depends on what just happened in a way that’s hard to centralize — e.g., a “writer” agent that sometimes needs the “researcher” again, sometimes needs the “editor,” and the choice depends on the specific content it just produced. It’s more flexible than Supervisor but harder to reason about and debug, because control flow isn’t centralized anywhere.

**3. Hierarchical Teams.** This is Supervisor, but recursively nested. A top-level supervisor doesn’t route directly to individual workers — it routes to *team supervisors*, and each team supervisor in turn routes to its own workers (or even to sub-team supervisors). Each “team” is typically implemented as its own subgraph (this is exactly why Chapter 9 came before this chapter).

*Analogy:* A large company’s org chart. The CEO doesn’t personally direct every engineer — the CEO routes work to a VP of Engineering and a VP of Marketing, and each VP routes work within their own department.

*When to use it:* When you have enough distinct specialists that a single flat Supervisor would need to choose between, say, 15 workers (hard for an LLM router to do reliably). Grouping related workers into teams, each with their own sub-supervisor, keeps each individual routing decision simpler (the top-level supervisor only picks among 3-4 teams; each team supervisor only picks among its own 3-4 workers).

**4. Swarm.** Similar in spirit to Network (no central dispatcher), but the mechanism for handing off control is standardized: each agent has access to special **handoff tools** — one tool per other agent it’s allowed to transfer control to (e.g., a `transferToBillingAgent` tool). The currently “active” agent decides on its own, by calling one of these tools, when to pass control to a specific teammate. The system tracks which agent is “active” so that on the *next* user turn, the conversation resumes with whichever agent was last active, rather than starting over from a default agent.

*Analogy:* A customer service call where the call doesn’t get routed by a separate menu system — instead, whichever rep currently has you on the line decides “let me transfer you to billing,” and clicks a literal “transfer” button (the handoff tool) for that one specific destination.

*When to use it:* Best when you want very natural, conversational hand-offs between a small number of peer agents (e.g., a “general support agent” handing off to a “billing agent” mid-conversation), and you want the *agents themselves* (not a separate router LLM call) to decide when a handoff is appropriate, based on the conversation so far. In LangGraph.js, this pattern is most easily built using the dedicated `@langchain/langgraph-swarm` package, which provides `createSwarm()` and `createHandoffTool()` helpers so you don’t have to hand-build the handoff-tool plumbing yourself.

## How routing actually happens in code: Command

You learned about the `Command` object back in Phase 2 (Control Flow) — it lets a node return both a state update *and* a routing decision (`goto`) in one object, instead of relying only on static `addConditionalEdges`. Multi-agent architectures lean on this heavily: each worker agent’s node, instead of always going to a fixed next node, returns something like:

```
return new Command({
  update: { messages: [aiResponse] },
  goto: "supervisor", // or another worker's name, in
Network/Swarm patterns
});
```

This is what lets a worker dynamically decide its own next step based on what just happened, rather than the graph's static edges deciding for it.

## Handoffs: what's actually being passed

A "handoff" is conceptually just: (1) a state update containing whatever context the next agent needs (typically the conversation history so far, sometimes filtered or summarized), plus (2) a routing decision about who receives it. In LangGraph this is almost always implemented as one Command return value combining both — there's no separate "handoff object" type, it's the same mechanism you already know.

## 10.2 Practical Lab

We'll build the same basic scenario — a system that handles either math questions or writing requests — across all four architectures, so you can directly compare how the same problem looks under each pattern.

### Lab 10.1 — Build a Supervisor that routes to a "math" or "writer" worker agent

#### Step 1 — Define shared state.

Create src/lab10-1.ts:

```
import "dotenv/config";
import { StateGraph, Annotation, START, END, Command,
MessagesAnnotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, AIMessage } from "@langchain/core/messages";
import { z } from "zod";

// We extend the built-in MessagesAnnotation (handles the messages
array + its reducer for us)
// with a field to track which agent should act next.
const SupervisorState = Annotation.Root({
  ...MessagesAnnotation.spec,
  next: Annotation<string>,
});
```

*Why MessagesAnnotation:* You met this in Phase 1 — it's a pre-built state shape for messages: BaseMessage[] with the correct "append, don't overwrite" reducer already wired up. Reusing it here saves us from re-defining a reducer we already know works.

#### Step 2 — Build the supervisor node using structured output.

```
const routerModel = new ChatOpenAI({ model: "gpt-4o-mini",
temperature: 0 });

const RouteSchema = z.object({
  next: z.enum(["math", "writer", "FINISH"]).describe(
    "Which worker should act next, or FINISH if the user's request
has been fully handled."
  ),
});

const routerModelWithSchema =
routerModel.withStructuredOutput(RouteSchema);

async function supervisorNode(state: typeof SupervisorState.State) {
```

```

const decision = await routerModelWithSchema.invoke([
  new HumanMessage(
    `You are a supervisor routing tasks to "math" (handles
calculations) or "writer" (handles writing/prose tasks) workers. ` +
    `Based on the conversation so far, decide who should act next,
or FINISH if the task is done.\n\n` +
    `Conversation:\n${state.messages.map(m => `${m.getType()}:
${m.content}`).join("\n")}`
  ),
]);

return new Command({
  goto: decision.next === "FINISH" ? END : decision.next,
  update: { next: decision.next },
});
}

```

*What withStructuredOutput is doing:* Instead of asking the model to write free-text like “I think the math worker should handle this” and then trying to parse that with string matching (fragile!), withStructuredOutput(RouteSchema) forces the model’s response to conform exactly to our Zod schema — we get back a guaranteed { next: "math" | "writer" | "FINISH" } object, no parsing required. This is the standard, reliable way to make an LLM make a *routing decision* rather than write prose.

*What the Command return is doing:* goto tells LangGraph which node to run next — directly, dynamically, from inside this node, rather than via a separate static conditional-edge function. update: { next: decision.next } also writes the decision into state, purely so we can inspect/log it; it’s not strictly required for routing to work, since goto already handles that.

### Step 3 — Build the two worker nodes.

```

const workerModel = new ChatOpenAI({ model: "gpt-4o-mini",
temperature: 0 });

async function mathNode(state: typeof SupervisorState.State) {
  const response = await workerModel.invoke([
    new HumanMessage(
      `You are a precise math assistant. Solve the user's
calculation and show your work briefly.\n\n` +
      state.messages.map(m => `${m.getType()}:
${m.content}`).join("\n")
    ),
  ]);

  return new Command({
    goto: "supervisor",    // workers always report back to the
supervisor
    update: { messages: [new AIMessage({ content: response.content,
name: "math" })] },
  });
}

async function writerNode(state: typeof SupervisorState.State) {
  const response = await workerModel.invoke([
    new HumanMessage(
      `You are a skilled writing assistant. Help with the user's
writing request.\n\n` +
      state.messages.map(m => `${m.getType()}:
${m.content}`).join("\n")
    ),
  ]);

  return new Command({

```



```

        goto: "supervisor",
        update: { messages: [new AIMessage({ content: response.content,
name: "writer" })] } },
    });
}

```

*Key detail* — *goto: "supervisor"*: This is the heart of the Supervisor pattern. No matter which worker ran, control *always* returns to the supervisor next, never directly to another worker, never straight to END. The supervisor alone decides whether the task is finished.

*Why name: "math" / name: "writer" on the AIMessage*: Tagging which agent produced which message makes the conversation log far easier to debug and display — especially once there are 3+ agents in play.

#### Step 4 — Wire the graph.

```

const supervisorGraph = new StateGraph(SupervisorState)
    .addNode("supervisor", supervisorNode, { ends: ["math", "writer",
END] })

    .addNode("math", mathNode, { ends: ["supervisor"] })
    .addNode("writer", writerNode, { ends: ["supervisor"] })
    .addEdge(START, "supervisor")
    .compile();

```

*Why the { ends: [...] } option*: Because routing decisions happen dynamically via `Command.goto` rather than static `addConditionalEdges()`, `LangGraph` needs you to *declare up front* the full set of possible destinations for each such node. This isn't optional boilerplate — it's what lets `LangGraph` validate your graph's structure and draw an accurate diagram, even though the *actual* destination is only decided at runtime.

*Notice there's no `addEdge("math", ...)` or `addEdge("writer", ...)`*: Because both worker nodes return their own `Command(goto: "supervisor")`, the routing is fully dynamic — we never declared a static edge for them, and we didn't need to.

#### Step 5 — Run it.

```

const result = await supervisorGraph.invoke({
    messages: [new HumanMessage("What is 847 * 23, and then write me a
one-sentence fun fact about that number?")],
});

console.log("\n--- Conversation ---");
for (const m of result.messages) {
    console.log(`[${m.getType()}]${(m as AIMessage).name ? " - " + (m
as AIMessage).name : ""}: ${m.content}`);
}

```

**Expected result**: Running `npx tsx src/lab10-1.ts` should show the supervisor routing first to `math` (to compute  $847 \times 23 = 19,481$ ), then back to the supervisor, then to `writer` (to produce the fun fact), then back to the supervisor, which finally decides FINISH.

**How to verify it worked correctly**: Print `result.next` — but more importantly, look at the printed conversation log and confirm you see *two* AI messages tagged with different name values (`math` and `writer`), proving both workers actually ran, in the right order, coordinated entirely by the supervisor.

**Common beginner mistakes**: - Forgetting the `{ ends: [...] }` option on nodes that return dynamic `Command` objects — `LangGraph` will throw a validation error at compile time telling you a node has no declared outgoing edges. - Having worker nodes route directly to END instead of back to "supervisor" — this breaks the Supervisor pattern,

because now the supervisor never gets a chance to decide if more work is needed. - Using free-text routing (“the model said ‘I think writer should go next’”) instead of withStructuredOutput — this works *sometimes* but fails unpredictably whenever the model phrases its answer slightly differently than your string-matching code expects.

---

## Lab 10.2 — Convert the supervisor system into a peer-to-peer Network

**Goal:** Remove the central supervisor and let math and writer decide for themselves whether to hand off to each other or finish.

### Step 1 — Reuse SupervisorState, but change how workers decide their own next step.

Create `src/lab10-2.ts`. We keep the same state shape, but now *each worker* — not a separate supervisor — makes the routing call.

```
import "dotenv/config";
import { StateGraph, Annotation, START, END, Command,
MessagesAnnotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, AIMessage } from "@langchain/core/messages";
import { z } from "zod";

const NetworkState = Annotation.Root({ ...MessagesAnnotation.spec
});

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0
});

const NextSchema = z.object({
  next: z.enum(["math", "writer", "FINISH"]).describe("Who should
act next, or FINISH if done."),
});
const modelWithRouting = model.withStructuredOutput(NextSchema);
```

*Why this is structurally different from Supervisor, even though the code looks similar:* In Lab 10.1, only *one* node (supervisorNode) ever made a routing decision. Here, *every* worker node makes its own routing decision after doing its work — there’s no single node whose entire job is “decide who goes next.” That’s the defining trait of Network: routing logic is distributed, not centralized.

### Step 2 — Build worker nodes that do their job AND decide the next step themselves.

```
async function mathNode(state: typeof NetworkState.State) {
  const answer = await model.invoke([
    new HumanMessage(`Solve this calculation, show your work
briefly:\n\n${state.messages.at(-1)?.content}`),
  ]);

  const routing = await modelWithRouting.invoke([
    new HumanMessage(
      `A math worker just produced this: "${answer.content}". ` +
      `Should "writer" now add something (e.g. a fun fact/sentence
about the result), or is the task FINISHED? ` +
      `Original request: "${state.messages[0].content}"`
    ),
  ]);

  return new Command({
    goto: routing.next === "FINISH" ? END : routing.next,
    update: { messages: [new AIMessage({ content: answer.content,
```

```

name: "math" }]] },
  });
}

  async function writerNode(state: typeof NetworkState.State) {
    const answer = await model.invoke([
      new HumanMessage(`Continue this conversation
helpfully:\n\n${state.messages.map(m => m.content).join("\n")}`),
    ]);

    const routing = await modelWithRouting.invoke([
      new HumanMessage(
        `A writer worker just produced this: "${answer.content}". Does
"math" still need to do anything, or is the task FINISHED?`
      ),
    ]);

    return new Command({
      goto: routing.next === "FINISH" ? END : routing.next,
      update: { messages: [new AIMessage({ content: answer.content,
name: "writer" }]] },
    });
  }
}

```

Notice the key structural difference from Lab 10.1: `mathNode` can route to `"writer"` *directly* — it never has to pass back through a central supervisor first. That’s a direct peer-to-peer handoff, which is the defining capability Network gives you that Supervisor doesn’t.

### Step 3 — Wire the graph (no supervisor node exists at all).

```

const networkGraph = new StateGraph(NetworkState)
  .addNode("math", mathNode, { ends: ["writer", END] })
  .addNode("writer", writerNode, { ends: ["math", END] })
  .addEdge(START, "math") // we still need ONE fixed entry point
  .compile();

const result = await networkGraph.invoke({
  messages: [new HumanMessage("What is 12 squared, and then write a
fun one-liner about that number?")],
});

for (const m of result.messages) {
  console.log(`[${(m as AIMessage).name ?? m.getType()}]:
${m.content}`);
}

```

Why we still need `addEdge(START, "math")`: Network removes the central *router*, not the need for an entry point — something has to be the first node the graph runs. Here we hardcode it to always start with `math`; in a more advanced version you could add a tiny “intake” node that decides the *first* worker dynamically, while still letting workers hand off to each other peer-to-peer afterward.

**Expected result:** Math computes 144, then hands off directly to writer (no supervisor in between), which adds a fun one-liner, then decides FINISH.

**Verification step:** Compare the number of total node-to-node “hops” in the printed conversation between this lab and Lab 10.1. In the Supervisor version, every worker-to-worker handoff costs *two* hops (worker → supervisor → next worker). In the Network version here, it costs *one* hop (worker → next worker directly). Counting hops is a good concrete way to see the architectural difference, not just read about it.

**Common beginner mistake:** Letting every worker route to every other worker without limiting { ends: [...] } correctly — in a Network with many agents, it's easy to accidentally create routing cycles where two agents keep handing off to each other indefinitely. Always sanity-check your ends declarations against the actual prompts you've given each router call, and consider adding a hop counter to state with a hard cutoff as a safety net (similar to recursionLimit, which you met in Phase 2).

---

## Lab 10.3 — Build a two-level hierarchical team (team leads + a top-level supervisor)

**Goal:** Group related workers into a “team,” led by its own mini-supervisor (built as a subgraph from Chapter 9), and have one top-level supervisor route between *teams*, not individual workers.

### Step 1 — Design the hierarchy.

We'll build: - A “**Research Team**” subgraph: its own internal supervisor routing between a search worker and a factcheck worker. - A “**Writing Team**” subgraph: its own internal supervisor routing between a draft worker and an edit worker. - A **top-level supervisor** that routes between the two *teams* (not the four individual workers).

This is exactly the org-chart analogy from the theory section, written in code.

### Step 2 — Build the Research Team as a subgraph (reusing the Supervisor pattern from Lab 10.1, scoped to one team).

Create src/lab10-3.ts:

```
import "dotenv/config";
import { StateGraph, Annotation, START, END, Command,
MessagesAnnotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, AIMessage } from "@langchain/core/messages";
import { z } from "zod";

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0
});

// --- Research Team ---
const ResearchTeamState = Annotation.Root({
...MessagesAnnotation.spec });

const ResearchRoute = z.object({ next: z.enum(["search",
"factcheck", "FINISH"]) });
const researchRouter = model.withStructuredOutput(ResearchRoute);

async function researchSupervisorNode(state: typeof
ResearchTeamState.State) {
  const decision = await researchRouter.invoke([
    new HumanMessage(
      `Route to "search" (find facts) or "factcheck" (verify a claim
already found), or FINISH if research is complete.\n\n` +
      state.messages.map(m => `${(m as AIMessage).name ??
m.getType()}: ${m.content}`).join("\n")
    ),
  ]);
  return new Command({ goto: decision.next === "FINISH" ? END :
decision.next });
}

async function searchWorkerNode(state: typeof
```

```

ResearchTeamState.State) {
    const response = await model.invoke([
        new HumanMessage(`Find 2-3 facts relevant to:
${state.messages[0].content}`),
    ]);
    return new Command({
        goto: "researchSupervisor",
        update: { messages: [new AIMessage({ content: response.content,
name: "search" })] },
    });
}

    async function factcheckWorkerNode(state: typeof
ResearchTeamState.State) {
        const response = await model.invoke([
            new HumanMessage(`Briefly verify or flag any questionable claims
in: ${state.messages.at(-1)?.content}`),
        ]);
        return new Command({
            goto: "researchSupervisor",
            update: { messages: [new AIMessage({ content: response.content,
name: "factcheck" })] },
        });
    }

    const researchTeam = new StateGraph(ResearchTeamState)
        .addNode("researchSupervisor", researchSupervisorNode, { ends:
["search", "factcheck", END] })
        .addNode("search", searchWorkerNode, { ends:
["researchSupervisor"] })
        .addNode("factcheck", factcheckWorkerNode, { ends:
["researchSupervisor"] })
        .addEdge(START, "researchSupervisor")
        .compile();

```

*What this is:* This is the exact same Supervisor pattern from Lab 10.1, just scoped to a smaller, single-purpose team and compiled into its own subgraph. Nothing new conceptually — this is Chapter 9 and the Supervisor pattern combined.

### Step 3 — Build the Writing Team the same way.

```

// --- Writing Team ---
const WritingTeamState = Annotation.Root({
    ...MessagesAnnotation.spec });

const WritingRoute = z.object({ next: z.enum(["draft", "edit",
"FINISH"]) });
const writingRouter = model.withStructuredOutput(WritingRoute);

    async function writingSupervisorNode(state: typeof
WritingTeamState.State) {
        const decision = await writingRouter.invoke([
            new HumanMessage(
                `Route to "draft" (write new content) or "edit" (polish
existing content), or FINISH when done.\n\n` +
                state.messages.map(m => `${(m as AIMessage).name ??
m.getType()}: ${m.content}`).join("\n")
            ),
        ]);
        return new Command({ goto: decision.next === "FINISH" ? END :
decision.next });
    }

    async function draftWorkerNode(state: typeof WritingTeamState.State)
{

```

```

        const response = await model.invoke([
            new HumanMessage(`Write a short first draft based on:
${state.messages.map(m => m.content).join("\n")}`),
        ]);
        return new Command({
            goto: "writingSupervisor",
            update: { messages: [new AIMessage({ content: response.content,
name: "draft" })] },
        });
    }

    async function editWorkerNode(state: typeof WritingTeamState.State)
    {
        const response = await model.invoke([
            new HumanMessage(`Lightly polish this draft for
clarity:\n\n${state.messages.at(-1)?.content}`),
        ]);
        return new Command({
            goto: "writingSupervisor",
            update: { messages: [new AIMessage({ content: response.content,
name: "edit" })] },
        });
    }

    const writingTeam = new StateGraph(WritingTeamState)
        .addNode("writingSupervisor", writingSupervisorNode, { ends:
["draft", "edit", END] })
        .addNode("draft", draftWorkerNode, { ends: ["writingSupervisor"]
})
        .addNode("edit", editWorkerNode, { ends: ["writingSupervisor"] })
        .addEdge(START, "writingSupervisor")
        .compile();

```

#### Step 4 — Build the top-level supervisor that routes between the two TEAMS.

```

// --- Top-level supervisor ---
const TopState = Annotation.Root({ ...MessagesAnnotation.spec });

const TopRoute = z.object({ next: z.enum(["researchTeam",
"writingTeam", "FINISH"]) });
const topRouter = model.withStructuredOutput(TopRoute);

async function topSupervisorNode(state: typeof TopState.State) {
    const decision = await topRouter.invoke([
        new HumanMessage(
            `Route to "researchTeam" (needs facts gathered/verified) or
"writingTeam" (needs content drafted/edited), ` +
            `or FINISH if the overall task is complete.\n\n` +
            state.messages.map(m => `${(m as AIMessage).name ??
m.getType()}: ${m.content}`).join("\n")
        ),
    ]);
    return new Command({ goto: decision.next === "FINISH" ? END :
decision.next });
}

// Wrapper nodes: each team subgraph has its OWN internal message
// accumulation,
// but here both team states happen to share the "messages" key
// shape with TopState,
// so we can pass the compiled subgraphs in directly (Situation A
// from Chapter 9)
// as long as we route back to the top supervisor afterward — which
// requires a thin wrapper.
async function researchTeamNode(state: typeof TopState.State) {

```

```

        const teamResult = await researchTeam.invoke({ messages:
state.messages });
        return new Command({ goto: "topSupervisor", update: { messages:
teamResult.messages } });
    }

    async function writingTeamNode(state: typeof TopState.State) {
        const teamResult = await writingTeam.invoke({ messages:
state.messages });
        return new Command({ goto: "topSupervisor", update: { messages:
teamResult.messages } });
    }

    const topGraph = new StateGraph(TopState)
        .addNode("topSupervisor", topSupervisorNode, { ends:
["researchTeam", "writingTeam", END] })
        .addNode("researchTeam", researchTeamNode, { ends:
["topSupervisor"] })
        .addNode("writingTeam", writingTeamNode, { ends: ["topSupervisor"]
})
        .addEdge(START, "topSupervisor")
        .compile();

```

*Why we used wrapper nodes here instead of adding researchTeam/writingTeam directly as nodes:* Even though the state shapes overlap (both use messages), each team subgraph ends at its own internal END, which doesn't automatically know to route back to "topSupervisor" afterward. The thin wrapper lets us explicitly .invoke() the team subgraph and then issue our own Command(goto: "topSupervisor"), giving us hierarchical control flow on top of automatic state merging. This is a nice middle ground between the two patterns from Chapter 9.

### Step 5 — Run it.

```

const result = await topGraph.invoke({
    messages: [new HumanMessage("Research one interesting fact about
octopuses, fact-check it, then write a 2-sentence blurb about
it.")],
});

console.log("\n--- Full hierarchy trace ---");
for (const m of result.messages) {
    console.log(`[${(m as AIMessage).name ?? m.getType()}]:
${m.content}`);
}

```

**Expected result:** The top supervisor routes to researchTeam, which internally bounces between its own researchSupervisor → search → researchSupervisor → factcheck → researchSupervisor → back out to topSupervisor. Then the top supervisor routes to writingTeam, which internally runs its own draft/edit loop, then returns. Finally the top supervisor decides FINISH.

**How to verify it worked correctly:** Run with { subgraphs: true } streaming (from Lab 9.3) instead of .invoke(), and confirm you can see all four worker nodes (search, factcheck, draft, edit) fire individually, nested two levels deep under their respective team nodes.

**Common beginner mistake:** Trying to make the top-level supervisor route directly to individual workers like "search" or "draft", defeating the entire purpose of hierarchy. The whole point is that the top supervisor's prompt only ever has to reason about 2-3 *teams*, not 6+ individual workers — keep that separation strict.

---



## Lab 10.4 — Implement a Swarm where agents hand off based on a handoff tool call

**Goal:** Use the dedicated swarm package to let two peer agents (a general support agent and a billing agent) decide for themselves, via tool calls, when to transfer the conversation.

### Step 1 — Install the swarm package.

```
npm install @langchain/langgraph-swarm
```

*Why a dedicated package instead of hand-building this like Lab 10.2's Network:* Swarm's defining feature — handoffs via *tool calls* that also correctly track “who's the active agent” across multiple separate user turns — involves enough fiddly state-tracking plumbing that the LangGraph team built a small dedicated package for it (`createSwarm`, `createHandoffTool`) so you don't have to reimplement that bookkeeping by hand.

### Step 2 — Create handoff tools.

Create `src/lab10-4.ts`:

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";
import { createReactAgent } from "@langchain/langgraph/prebuilt";
import { createSwarm, createHandoffTool } from
"@langchain/langgraph-swarm";
import { MemorySaver } from "@langchain/langgraph";
import { HumanMessage } from "@langchain/core/messages";

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0
});

// A handoff tool is just a special tool: calling it doesn't "do
work,"
// it signals "transfer control to this other named agent."
const transferToBilling = createHandoffTool({
  agentName: "billingAgent",
  description: "Transfer the conversation to the billing agent for
payment, refund, or invoice questions.",
});

const transferToSupport = createHandoffTool({
  agentName: "supportAgent",
  description: "Transfer the conversation back to general support
for non-billing questions.",
});
```

*Why this counts as a “tool” at all:* From the model's point of view, a handoff tool looks exactly like any other tool it could call (you'll learn the general tool-calling mechanism properly in Chapter 11). The model decides, based on the conversation, “this sounds like a billing question, I should call `transferToBilling`” — the same way it might decide to call a `getWeather` tool. The *effect* of calling it (routing control elsewhere) is handled by the swarm package behind the scenes.

### Step 3 — Build each agent using `createReactAgent` (a prebuilt agent — don't worry, Chapter 11 explains this fully; for now, treat it as “an agent with these tools and this prompt”).

```
const supportAgent = createReactAgent({
  llm: model,
  tools: [transferToBilling],
  prompt: "You are a friendly general support agent. Handle general
questions yourself. " +
    "If the user asks about billing, payments, refunds, or
```



```

invoices, transfer to the billing agent.",
  name: "supportAgent",
});

const billingAgent = createReactAgent({
  llm: model,
  tools: [transferToSupport],
  prompt: "You are a billing specialist. Handle payment, refund, and
invoice questions. " +
          "If the user asks something unrelated to billing, transfer
back to general support.",
  name: "billingAgent",
});

```

Why each agent only has the handoff tool to the other\* agent, not to itself:\* An agent transferring to itself doesn't make sense, so we only wire up the one relevant outgoing handoff per agent. In a swarm with more than two agents, each agent would typically get one handoff tool per teammate it's allowed to transfer to.

#### Step 4 — Assemble the swarm.

```

const checkpointer = new MemorySaver();

const swarm = createSwarm({
  agents: [supportAgent, billingAgent],
  defaultActiveAgent: "supportAgent",
}).compile({ checkpointer });

```

What `defaultActiveAgent` means: On a brand-new conversation thread, before any handoff has happened, this is who answers first. Why we need a `checkpointer` here, specifically: Swarm's "remember who's currently active" behavior across multiple separate `.invoke()` calls (i.e., multiple turns of a real conversation) depends on persisted state — exactly the `thread_id`-based persistence mechanism you learned in Phase 3. Without a `checkpointer`, every `.invoke()` would reset back to `defaultActiveAgent`, defeating the purpose.

#### Step 5 — Run a multi-turn conversation and watch the active agent change.

```

const threadConfig = { configurable: { thread_id: "swarm-demo-1" }
};

const turn1 = await swarm.invoke(
  { messages: [new HumanMessage("Hi, what are your support hours?")]
},
  threadConfig
);
console.log("Turn 1 last message:", turn1.messages.at(-1)?.content);

const turn2 = await swarm.invoke(
  { messages: [new HumanMessage("Actually, I was charged twice for
my last invoice, can you help?")] },
  threadConfig
);
console.log("Turn 2 last message:", turn2.messages.at(-1)?.content);

const turn3 = await swarm.invoke(
  { messages: [new HumanMessage("Got it, thanks. One more general
question — do you have a mobile app?")] },
  threadConfig
);
console.log("Turn 3 last message:", turn3.messages.at(-1)?.content);

```

**Expected result:** Turn 1 is answered by supportAgent (general hours question, no handoff needed). Turn 2 triggers a handoff — supportAgent calls transferToBilling, and the response comes from billingAgent. Turn 3, despite being a *new* .invoke() call, should still be picked up by billingAgent if it's billing-adjacent, or hand back to supportAgent since it's clearly a general question — because the *same* thread\_id means the swarm remembers which agent was last active and resumes from there, rather than defaulting back to supportAgent every time.

**How to verify it worked correctly:** Inspect the full message list (turn2.messages) and look for a tool call message invoking transferToBilling between the support agent's response and the billing agent's response — that's the concrete evidence a handoff actually occurred, not just that the *content* happened to sound billing-related.

**Common beginner mistakes:** - Forgetting the checkpointer + thread\_id, then being confused why every new .invoke() “resets” to the default agent — without persistence, Swarm has no memory of a previous handoff. - Writing handoff tool descriptions that are too vague (e.g., just "Transfer to billing" with no guidance on *when*). The model's handoff decision quality is only as good as how clearly the tool's description field explains when to use it — treat it with the same care as a system prompt.

## 10.3 Knowledge Check — Multi-Agent Architectures

### Summary

Multi-agent systems split work across multiple narrowly-prompted LLM nodes instead of one overloaded agent, improving focus, tool accuracy, and debuggability. **Supervisor** routes all traffic through one central dispatcher node, with workers always reporting back to it. **Network** removes the central dispatcher — agents hand off directly to each other based on their own judgment. **Hierarchical Teams** nests Supervisor recursively: a top-level supervisor routes between team subgraphs, each with its own internal mini-supervisor. **Swarm** uses dedicated handoff tools so agents themselves decide, via tool calls, when to transfer control to a named teammate, with the currently-active agent persisted across turns via a checkpointer. All four patterns are built on the same two primitives you already knew before this chapter: Command (for dynamic state update + routing) and subgraphs (for grouping related logic).

### Key Takeaways

- Supervisor = centralized routing; workers always return to the supervisor, never to each other directly.
- Network = decentralized routing; any agent can hand off to any other agent directly.
- Hierarchical Teams = Supervisor nested recursively, using subgraphs as “teams,” to keep each individual routing decision simple even as the total number of workers grows.
- Swarm = handoffs expressed as tool calls the agent itself decides to invoke, with active-agent state persisted via a checkpointer across separate .invoke() calls.
- withStructuredOutput with a Zod enum schema is the reliable way to get a routing decision from an LLM — avoid parsing free text for routing.
- Nodes that return dynamic Command(goto: ...) must declare their possible destinations via the { ends: [...] } option when added to

the graph.

## Practice Questions

1. In the Supervisor pattern, why does every worker route back to the supervisor instead of straight to END?
2. What specific problem does Hierarchical Teams solve that a flat Supervisor with 10 workers would struggle with?
3. In Swarm, what does the checkpoint's `thread_id` actually preserve between two separate `.invoke()` calls?
4. Why is `withStructuredOutput` preferred over asking the model to "just say the next agent's name in plain text" for routing decisions?

## Exercises

1. Add a third worker to the Lab 10.1 Supervisor system — a "researcher" agent — and update the supervisor's routing schema and prompt to include it.
2. In the Lab 10.2 Network, add a hop counter to state (an `Annotation<number>` incremented on every worker call) and have each worker's routing prompt include "if `hopCount > 4`, you must choose FINISH" to prevent runaway handoff loops.

## Mini Project

Build a **three-team Hierarchical system**: Research Team (from Lab 10.3), Writing Team (from Lab 10.3), and a new **Review Team** (two workers: a `factCheck` worker and a `toneCheck` worker, with its own mini-supervisor) that the top-level supervisor can route to *after* the Writing Team finishes, before declaring FINISH. Confirm with subgraphs: true streaming that you can trace a request through all three teams in sequence.

---

# Chapter 11 — Tools & Prebuilt Components

## 11.1 Concept and Theory

### What is a "tool," conceptually?

An LLM, on its own, can only do one thing: take in text and produce text out. It can't actually look up today's weather, run a calculation it isn't confident about, query a database, or send an email — it can only *talk about* doing those things. A **tool** bridges that gap: it's a regular function (you write it, in plain TypeScript) that the model can request to have executed, by name, with specific arguments, as part of generating its response.

The flow is always the same shape: you describe a tool to the model (its name, what it does, what arguments it takes). The model, when it decides a tool would help answer the current request, doesn't run the function itself — it outputs a special structured message saying "please call `getWeather` with { `city`: "Paris" }." Your code then actually executes that function, gets a real result, and feeds that result *back* to the model so it can use it to write its final answer.

### Why this two-step dance instead of just calling functions directly?

Because the model decides **which** tool to call and **with what arguments**, based on the conversation — that’s the whole value. You’re not hardcoding “always call `getWeather`”; you’re giving the model a *capability* and letting it decide, contextually, whether and how to use it. This is what separates a “tool-using agent” from a plain script that calls a fixed sequence of functions.

## Defining a tool

In `LangChain.js`, you define a tool with the `tool()` helper, giving it a function, a name, a description, and a Zod schema describing its arguments:

```
import { tool } from "@langchain/core/tools";
import { z } from "zod";

const getWeather = tool(
  async ({ city }) => {
    // pretend this calls a real weather API
    return `It's sunny and 22°C in ${city}.`;
  },
  {
    name: "getWeather",
    description: "Get the current weather for a given city.",
    schema: z.object({ city: z.string().describe("The city to get weather for") }),
  }
);
```

Three things matter enormously here, and beginners often underrate all three: - **name** must be something the model can reliably reference — keep it short and unambiguous. - **description** is, functionally, a tiny prompt. The model decides *whether* to call this tool based almost entirely on how well the description matches the user’s actual need. A vague description (“does stuff with cities”) leads to the model either never calling it or calling it at the wrong times. - **schema** (the Zod object) is both a strict contract for what arguments look like *and* extra guidance for the model — the `.describe()` calls on individual fields help the model fill in arguments correctly.

## The tool-calling loop pattern

Once a model is bound to one or more tools (via `model.bindTools([...])`), here’s the actual step-by-step loop that happens, and which you, as the graph author, are responsible for wiring up node-by-node:

1. **Agent node:** the model receives the conversation so far and decides either (a) it has enough information to respond directly, in which case it returns a normal text answer, or (b) it needs a tool, in which case it returns an `AIMessage` containing one or more `tool_calls` (structured requests: tool name + arguments) instead of a plain text answer.
2. **Routing:** your graph checks — does the latest `AIMessage` contain `tool_calls`? If yes, route to a tool-execution node. If no, the conversation is done; route to `END`.
3. **Tool execution node (ToolNode):** this special, prebuilt node looks at the pending tool calls, actually runs the matching JavaScript functions with the given arguments, and wraps each result in a `ToolMessage` (a message type specifically for carrying tool results back into the conversation).
4. **Back to the agent node:** the model sees the new `ToolMessage(s)` added to the conversation and decides again — does it now have enough to answer, or does it need to call another tool? This loop can repeat multiple times for complex, multi-step tasks.

This is sometimes visualized as a simple 2-node cycle: **agent** → (**if tool call**) → **tools** → **agent** → (**if tool call**) → **tools** → **agent** → (**no tool call**) → **END**.

### **ToolNode: don't hand-write the execution step yourself**

You could, in principle, write your own node that manually reads `tool_calls` off the last message, looks up the right function, calls it, and wraps the result in a `ToolMessage`. But this is exactly the kind of repetitive, easy-to-get-subtly-wrong plumbing that LangGraph ships a prebuilt solution for: `ToolNode`. You give it a list of your tool definitions, and it handles matching tool-call names to the right function, executing them (including in parallel if the model requested multiple tool calls at once), error handling when a tool throws, and correctly formatting each result as a `ToolMessage`. Unless you have a very unusual requirement, always reach for `ToolNode` rather than reimplementing this loop by hand.

### **createReactAgent: don't hand-write the whole loop either**

Once you understand the agent → tools → agent cycle, you'll notice that an enormous fraction of real-world agents are *exactly* this pattern, with no extra customization needed: bind some tools to a model, loop between an agent node and a `ToolNode` until the model stops requesting tools, then return the final answer. Because this shape is so common, LangGraph provides `createReactAgent()` — a single function call that builds, wires, and compiles this entire 2-node graph for you (agent node with tools bound, `ToolNode`, conditional routing between them, all included).

"ReAct" here refers to the well-known prompting pattern of interleaving **Reasoning** and **Acting** — the model reasons about what to do, acts (calls a tool), observes the result, and reasons again. `createReactAgent` is literally a packaged implementation of that loop, which is also exactly the loop the Supervisor/Network workers in Chapter 10 used internally (you actually already used `createReactAgent` once, in Lab 10.4's Swarm example, without it being explained yet — now you know what it was doing).

### **When to use a prebuilt agent vs. a hand-rolled custom graph**

Use `createReactAgent` when: - Your agent's behavior really is "reason, optionally call tools, repeat until done, then answer" with no extra custom steps. - You want to get something working quickly, and customize later only if needed.

Build a custom graph by hand (agent node + `ToolNode` + conditional edge, wired yourself) when: - You need extra nodes in the loop that aren't just "agent" and "tools" — e.g., a human-approval step before certain tool calls (recall Phase 4's human-in-the-loop interrupts), a validation/guardrail node, or logic that depends on *which specific tool* was called, not just "a tool was called." - You need a state schema with extra fields beyond messages (e.g., tracking a running cost counter, a user ID, custom business state) that the prebuilt agent's default state shape doesn't include — although note `createReactAgent` does support a custom stateSchema, covered next. - You need multiple different conditional branches in the loop, not a single binary "tool call or not" check.

### **Customizing prebuilt agents**

`createReactAgent` isn't all-or-nothing — it accepts several customization points without forcing you to abandon it and hand-roll everything: - **prompt** — a system prompt string (or a function for more dynamic prompting) that shapes how the agent reasons and when it reaches for tools. - **stateSchema** — swap in your own custom state annotation (as long as it includes a `messages` field) if you need extra fields tracked alongside the conversation. - **Hooks** (e.g., a pre-model hook) — a function that runs *before* the agent node calls the model each time, useful for things like input validation, trimming an overly-long message history, or injecting extra context just-in-time.

These customization points cover a surprisingly large fraction of “I thought I’d need a fully custom graph” situations — it’s worth reaching for the customized prebuilt agent before defaulting to a hand-rolled one.

## 11.2 Practical Lab

### Lab 11.1 — Wire up a `ToolNode` with two custom tools in a tool-calling loop

**Goal:** Build the agent → tools → agent loop by hand, so you fully understand what `createReactAgent` automates before you start relying on it.

#### Step 1 — Define two simple tools.

Create `src/lab11-1.ts`:

```
import "dotenv/config";
import { tool } from "@langchain/core/tools";
import { z } from "zod";
import { ChatOpenAI } from "@langchain/openai";
import { StateGraph, MessagesAnnotation, START, END } from
"@langchain/langgraph";
import { ToolNode } from "@langchain/langgraph/prebuilt";
import { HumanMessage, AIMessage } from "@langchain/core/messages";

const getWeather = tool(
  async ({ city }: { city: string }) => {
    const fakeData: Record<string, string> = {
      paris: "15°C, light rain",
      tokyo: "28°C, clear skies",
      "new york": "19°C, partly cloudy",
    };
    return fakeData[city.toLowerCase()] ?? `No weather data found
for ${city}.`;
  },
  {
    name: "getWeather",
    description: "Get the current weather for a specific city.",
    schema: z.object({ city: z.string().describe("The city name,
e.g. 'Paris'") }),
  }
);

const calculate = tool(
  async ({ expression }: { expression: string }) => {
    // A real implementation would use a safe math parser, not eval.
    // We keep this lab focused on the graph mechanics, not math-
    parsing safety.
    try {
      // eslint-disable-next-line no-eval
      const result = eval(expression);
```

```

        return `Result: ${result}`;
      } catch {
        return `Could not evaluate expression: ${expression}`;
      }
    },
    {
      name: "calculate",
      description: "Evaluate a basic arithmetic expression, e.g. '12 * 7' or '(4 + 5) / 3'.",
      schema: z.object({ expression: z.string().describe("A basic arithmetic expression") }),
    }
  );

  const tools = [getWeather, calculate];

```

*Why two tools instead of one:* With only one tool available, you can't really observe the model's *selection* behavior — it either calls the one tool or it doesn't. With two clearly different tools, you can verify the model picks the *right* one for a given question, which is the actual skill being tested here.

*Note on eval:* We use it only to keep this lab's focus on graph wiring rather than writing a safe expression parser. Never use raw eval on untrusted input in a real application — for production, use a dedicated math expression library.

## Step 2 — Bind the tools to the model.

```

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

const modelWithTools = model.bindTools(tools);

```

*What .bindTools() does:* It returns a new model instance that, on every call, includes the tool definitions (names, descriptions, schemas) as part of the request to the LLM API. The model provider's API uses this to decide when to emit a `tool_calls` field instead of plain text. Without this binding step, the model has no idea these tools even exist.

## Step 3 — Build the agent node.

```

async function agentNode(state: typeof MessagesAnnotation.State) {
  const response = await modelWithTools.invoke(state.messages);
  return { messages: [response] };
}

```

*What's happening:* This node just sends the full conversation history to the tool-bound model and appends whatever it returns (which might be a plain answer, or might be an `AIMessage` containing `tool_calls`) to the message list. Note this node does *not* contain any logic about whether to call tools — that decision lives entirely inside the LLM call itself, based on the bound tool definitions.

## Step 4 — Add ToolNode and the routing logic.

```

const toolNode = new ToolNode(tools);

function shouldContinue(state: typeof MessagesAnnotation.State):
"tools" | typeof END {
  const lastMessage = state.messages.at(-1) as AIMessage;
  if (lastMessage.tool_calls && lastMessage.tool_calls.length > 0) {
    return "tools";
  }
  return END;
}

```

*What ToolNode does internally, in plain terms:* It looks at the most

recent `AIMessage`'s `tool_calls` array, and for each one, finds the tool in the list you gave it whose name matches, runs that tool's function with the provided arguments, and produces a `ToolMessage` with the result — all without you writing any of that matching/execution/wrapping logic yourself.

*What shouldContinue does:* This is the conditional-edge routing function (familiar from Phase 2). It inspects the latest message: if it has pending tool calls, route to "tools"; otherwise, the model gave a final answer, so route to END.

### Step 5 — Wire the loop.

```
const agentGraph = new StateGraph(MessagesAnnotation)
  .addNode("agent", agentNode)
  .addNode("tools", toolNode)
  .addEdge(START, "agent")
  .addConditionalEdges("agent", shouldContinue, { tools: "tools",
[END]: END })
  .addEdge("tools", "agent") // <-- this is what makes it a LOOP
  .compile();
```

*Why addEdge("tools", "agent") is the most important line in this lab:* After a tool runs, control goes *back* to the agent node, not to END. This is what allows multi-step tool use — the model can call one tool, see the result, decide it needs a second tool, call that, see its result, and only then produce a final answer. Without this edge looping back, the agent could only ever call one tool and then the graph would have nowhere to send it next.

### Step 6 — Run it with a question requiring both tools.

```
const result = await agentGraph.invoke({
  messages: [new HumanMessage("What's the weather in Tokyo, and what
is 234 * 18?")],
});

for (const m of result.messages) {
  console.log(`[${m.getType()}]`, m.content || (m as
AIMessage).tool_calls);
}
```

**Expected result:** You should see the conversation trace show: a `HumanMessage` (your question), an `AIMessage` with `tool_calls` (possibly two at once — `getWeather` and `calculate`), one or two `ToolMessages` with the actual results, and finally a plain-text `AIMessage` combining both answers (Tokyo's weather and 4212).

**How to verify it worked correctly:** Count the `ToolMessages` in the printed trace — there should be exactly two (one per tool called), and their content should contain the real computed values ("28°C, clear skies" and "Result: 4212"), not placeholder text. If you see only an `AIMessage` with `tool_calls` and no follow-up `ToolMessage`, that means the `tools → agent` loop edge is missing or misconfigured.

**Common beginner mistakes:**

- Forgetting `addEdge("tools", "agent")` entirely — the graph then runs the tool once but never lets the model see the result or produce a final answer; `.invoke()` will appear to finish "successfully" but the result is just the raw tool call, with no final response.
- Not binding tools with `.bindTools()` and expecting the model to call them anyway — without binding, the model has no idea the tools exist and will just try to answer the math/weather question from its own (often wrong, for math) reasoning instead.
- Mismatched tool name strings between the tool definition and what you expect in routing/debug code — names must match exactly for `ToolNode` to find the right function.

---



## Lab 11.2 — Build the same agent using createReactAgent() and compare code size

**Goal:** Rebuild Lab 11.1's exact capability using the prebuilt agent, and see directly how much boilerplate it removes.

### Step 1 — The entire agent, in one call.

Create src/lab11-2.ts:

```
import "dotenv/config";
import { createReactAgent } from "@langchain/langgraph/prebuilt";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, AIMessage } from "@langchain/core/messages";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

const getWeather = tool(
  async ({ city }: { city: string }) => {
    const fakeData: Record<string, string> = {
      paris: "15°C, light rain",
      tokyo: "28°C, clear skies",
      "new york": "19°C, partly cloudy",
    };
    return fakeData[city.toLowerCase()] ?? `No weather data found for ${city}.`;
  },
  {
    name: "getWeather",
    description: "Get the current weather for a specific city.",
    schema: z.object({ city: z.string().describe("The city name, e.g. 'Paris'") }),
  }
);

const calculate = tool(
  async ({ expression }: { expression: string }) => {
    try {
      // eslint-disable-next-line no-eval
      return `Result: ${eval(expression)}`;
    } catch {
      return `Could not evaluate expression: ${expression}`;
    }
  },
  {
    name: "calculate",
    description: "Evaluate a basic arithmetic expression, e.g. '12 * 7' or '(4 + 5) / 3'."
    schema: z.object({ expression: z.string().describe("A basic arithmetic expression") }),
  }
);

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

const prebuiltAgent = createReactAgent({
  llm: model,
  tools: [getWeather, calculate],
});

const result = await prebuiltAgent.invoke({
  messages: [new HumanMessage("What's the weather in Tokyo, and what is 234 * 18?")],
});
```

```

    for (const m of result.messages) {
      console.log(`[${m.getType()}]`, m.content || (m as
AIMessage).tool_calls);
    }

```

**What to notice — the size comparison:** Lab 11.1 required: a manually-defined state-routing function (shouldContinue), a manual agentNode, manually constructing a StateGraph, manually adding both nodes, manually wiring the conditional edge AND the loop-back edge. Lab 11.2 replaces all of that with a single createReactAgent({ llm, tools }) call — same tool-binding, same agent/tools/agent loop, same routing logic, fully handled internally.

**Expected result:** Functionally identical output to Lab 11.1 — Tokyo’s weather and 4212, combined into a final natural-language answer.

**Verification step:** Run both src/lab11-1.ts and src/lab11-2.ts with the exact same question and confirm the final AIMessage content is materially equivalent (exact wording will differ slightly between runs due to normal LLM variability, but both should correctly report Tokyo’s weather and the value 4212).

**When this comparison matters in practice:** If your agent genuinely only needs “reason, call tools, repeat, answer” with no extra steps, hand-rolling the Lab 11.1 version provides no benefit over Lab 11.2 — it’s strictly more code to maintain for the same behavior. Reach for the hand-rolled version only once you have a concrete reason (see the “when to use a custom graph” theory section above).

## Lab 11.3 — Customize a prebuilt agent’s system prompt and state schema

**Goal:** Show that createReactAgent isn’t a black box — you can shape its behavior and extend its state without abandoning it for a fully custom graph.

### Step 1 — Add a custom system prompt.

Create src/lab11-3.ts, reusing the tools from Lab 11.2:

```

import "dotenv/config";
import { createReactAgent } from "@langchain/langgraph/prebuilt";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, AIMessage } from "@langchain/core/messages";
import { tool } from "@langchain/core/tools";
import { z } from "zod";
import { Annotation, MessagesAnnotation } from
"@langchain/langgraph";

const getWeather = tool(
  async ({ city }: { city: string }) => `It's sunny and 22°C in
${city}.`,
  {
    name: "getWeather",
    description: "Get the current weather for a specific city.",
    schema: z.object({ city: z.string() }),
  }
);

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0
});

const pirateAgent = createReactAgent({
  llm: model,
  tools: [getWeather],

```

```

    prompt: "You are a friendly pirate-themed weather assistant. " +
      "Always answer in pirate speak ('arr', 'ye', 'matey'), but
      still use the getWeather tool to get real data before answering.",
  });

  const pirateResult = await pirateAgent.invoke({
    messages: [new HumanMessage("What's the weather in Paris?")],
  });
  console.log("Pirate agent:", pirateResult.messages.at(-1)?.content);

```

*Why this matters:* The prompt option is how you shape *tone*, *persona*, and *reasoning style* without touching the underlying agent/tool loop at all. This is the single most common customization you'll reach for.

## Step 2 — Extend the state schema with a custom field.

```

// Custom state: messages (required by createReactAgent) + a
// userTier field
// the agent's tools or pre-model hook could use to adjust behavior.
const CustomAgentState = Annotation.Root({
  ...MessagesAnnotation.spec,
  userTier: Annotation<"free" | "pro">,
});

const tieredAgent = createReactAgent({
  llm: model,
  tools: [getWeather],
  stateSchema: CustomAgentState,
  prompt: (state) =>
    state.userTier === "pro"
      ? "You are a premium weather assistant. Give detailed,
        thorough answers."
      : "You are a basic weather assistant. Keep answers to one
        short sentence.",
});

const freeResult = await tieredAgent.invoke({
  messages: [new HumanMessage("What's the weather in Paris?")],
  userTier: "free",
});
console.log("\nFree tier:", freeResult.messages.at(-1)?.content);

const proResult = await tieredAgent.invoke({
  messages: [new HumanMessage("What's the weather in Paris?")],
  userTier: "pro",
});
console.log("Pro tier:", proResult.messages.at(-1)?.content);

```

*What's happening with prompt as a function:* Instead of a fixed string, prompt can be a function of the current state, letting you dynamically shape the system prompt per-request based on custom fields you added to stateSchema — here, userTier, which has nothing to do with the built-in messages field, demonstrating the agent's state isn't locked to only what createReactAgent ships with by default.

**Expected result:** The free-tier response is noticeably shorter than the pro-tier response, even though both used the identical getWeather tool and the identical underlying question — proving the custom state field genuinely influenced agent behavior.

**Verification step:** Compare

freeResult.messages.at(-1)?.content.length against  
 proResult.messages.at(-1)?.content.length — the pro response should reliably be longer, given the explicit prompt instruction difference.

**Common beginner mistake:** Forgetting that stateSchema must still include a messages-shaped field (which is why we spread ...MessagesAnnotation.spec rather than defining CustomAgentState from scratch) — createReactAgent relies on that field internally to manage the conversation loop, and omitting it will cause errors.

---

## Lab 11.4 — Add a pre-model hook to a prebuilt agent for input validation

**Goal:** Use a hook to run custom logic — here, simple input validation/sanitization — every time, right before the model is called, without writing a fully custom graph.

### Step 1 — Define a pre-model hook function.

Create src/lab11-4.ts:

```
import "dotenv/config";
import { createReactAgent } from "@langchain/langgraph/prebuilt";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, AIMessage, RemoveMessage } from
"@langchain/core/messages";
import { tool } from "@langchain/core/tools";
import { z } from "zod";
import { MessagesAnnotation } from "@langchain/langgraph";

const getWeather = tool(
  async ({ city }: { city: string }) => `It's sunny and 22°C in
${city}.`,
  { name: "getWeather", description: "Get weather for a city.",
schema: z.object({ city: z.string() }) }
);

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0
});

// A pre-model hook runs right before each call to the LLM inside
the agent loop.
// Here we use it to trim conversation history to the last 6
messages,
// preventing unbounded context growth in long-running
conversations.
function trimHistoryHook(state: typeof MessagesAnnotation.State) {
  const MAX_MESSAGES = 6;
  if (state.messages.length <= MAX_MESSAGES) {
    return {}; // no change needed
  }

  const excess = state.messages.slice(0, state.messages.length -
MAX_MESSAGES);
  return {
    messages: excess.map((m) => new RemoveMessage({ id: m.id! })),
  };
}
```

*Why a hook instead of a separate graph node:* You could build this as its own node in a fully custom graph, wired in before the agent node. But for a behavior this small and generic (“always do X right before calling the model, every single loop iteration”), a hook keeps you inside the simpler createReactAgent setup rather than forcing a full rebuild. This is a perfect example of the “customization points cover more than you’d expect” point from the theory section.

*What RemoveMessage does:* This is a special message type whose entire purpose is “tell the reducer to delete the message with this id from state,” rather than adding a new message. Returning a list of RemoveMessage objects from a node (or hook) is the standard LangGraph pattern for trimming or pruning conversation history, since the default messages reducer normally only knows how to *append*.

### Step 2 — Wire the hook into the agent.

```
const trimmedAgent = createReactAgent({
  llm: model,
  tools: [getWeather],
  preModelHook: trimHistoryHook,
});
```

### Step 3 — Simulate a long conversation and confirm trimming happens.

```
let state = { messages: [new HumanMessage("Hi, I'm planning a
trip.")] };
state = await trimmedAgent.invoke(state) as typeof state;

const fillerTurns = [
  "I like museums.",
  "I also like good food.",
  "What's the weather in Paris?",
  "And in Tokyo?",
  "One more: and in New York?",
];

for (const turn of fillerTurns) {
  state.messages.push(new HumanMessage(turn));
  state = await trimmedAgent.invoke(state) as typeof state;
  console.log(`After "${turn}" -> message count in state:
${state.messages.length}`);
}
```

**Expected result:** As the conversation grows past 6 messages, you should see the printed message count stabilize (rather than growing unboundedly with every turn), because the hook is removing older messages before each model call.

**How to verify it worked correctly:** Print `state.messages.map(m => m.content)` after the final turn and confirm the earliest messages (e.g., “Hi, I’m planning a trip.”) are no longer present, while the most recent ones are.

**Common beginner mistakes:** - Returning a plain new array of trimmed messages from the hook (e.g., `{ messages: state.messages.slice(-6) }`) instead of RemoveMessage objects. Because the default reducer *appends*, this would actually make the conversation grow even larger (it would append a second copy of the last 6 messages) rather than trimming it. Always use RemoveMessage when the goal is deletion. - Forgetting that `m.id` must exist on each message for RemoveMessage to target it correctly — messages constructed via `new HumanMessage(...)` etc. are automatically assigned an id, so this isn’t usually a problem, but it’s worth being aware of if you ever build messages manually without going through the standard message classes.

## 11.3 Knowledge Check — Tools & Prebuilt Components

## Summary

A tool is a regular function the model can request to have executed, by name and arguments, via `tool()` with a Zod schema describing its inputs. The model never runs the function itself — it emits a `tool_calls` request, your graph routes to a `ToolNode` (which matches, executes, and wraps results as `ToolMessages`), and control loops back to the agent node so the model can use the result. `createReactAgent` packages this entire agent/tools/agent loop into one function call, and remains customizable via `prompt`, `stateSchema`, and hooks (like a pre-model hook) — covering the large majority of real-world agent needs without requiring a hand-rolled graph.

## Key Takeaways

- A tool's description and the `.describe()` calls in its Zod schema function as a mini-prompt that directly determines whether and how correctly the model uses it.
- `model.bindTools([...])` is required before a model will ever emit `tool_calls` for those tools.
- `ToolNode` automates matching tool calls to functions, executing them, and wrapping results as `ToolMessages` — don't hand-write this.
- The loop-back edge (tools → agent) is what allows multi-step tool use; omitting it breaks the agent after one tool call.
- `createReactAgent` builds the identical agent/tools/agent loop in one call; use it by default, and only hand-roll a custom graph when you have a concrete extra requirement (extra nodes, multiple branches, etc.).
- `prompt`, `stateSchema`, and hooks (e.g. a pre-model hook) let you customize a prebuilt agent's persona, extra state fields, and per-call preprocessing, respectively, without abandoning `createReactAgent`.
- `RemoveMessage` is the correct mechanism for deleting messages from state; appending a "trimmed" list does not work with the default reducer.

## Practice Questions

1. What two pieces of information does a `tool_calls` entry on an `AIMessage` actually contain?
2. Why won't a model call a tool it technically "knows about" in your codebase if you forgot `.bindTools()`?
3. What's the concrete failure mode if you forget the `addEdge("tools", "agent")` loop-back edge?
4. Name two customization points `createReactAgent` supports without requiring a fully custom graph.

## Exercises

1. Add a third tool (e.g., a `convertCurrency` tool) to the Lab 11.2 prebuilt agent, and ask a question that requires chaining all three tools together in one request (weather, math, and currency conversion).
2. Modify the Lab 11.4 pre-model hook so that instead of a fixed message count, it keeps trimming until the *total character count* of remaining messages is under some threshold — a more realistic approximation of staying under a token budget.

## Mini Project

Build a “trip planner” **prebuilt agent** with three tools (getWeather, searchAttractions — simulate with an LLM call like in Lab 9.1’s searchNode, and convertCurrency), a custom prompt giving it a clear persona, a stateSchema extended with a tripBudget: number field, and a pre-model hook that trims history to the last 8 messages. Confirm it can handle a multi-part request like “What’s the weather in Lisbon, find me 2 attractions there, and convert a \$50 budget to euros” in a single .invoke() call, correctly chaining all three tools.

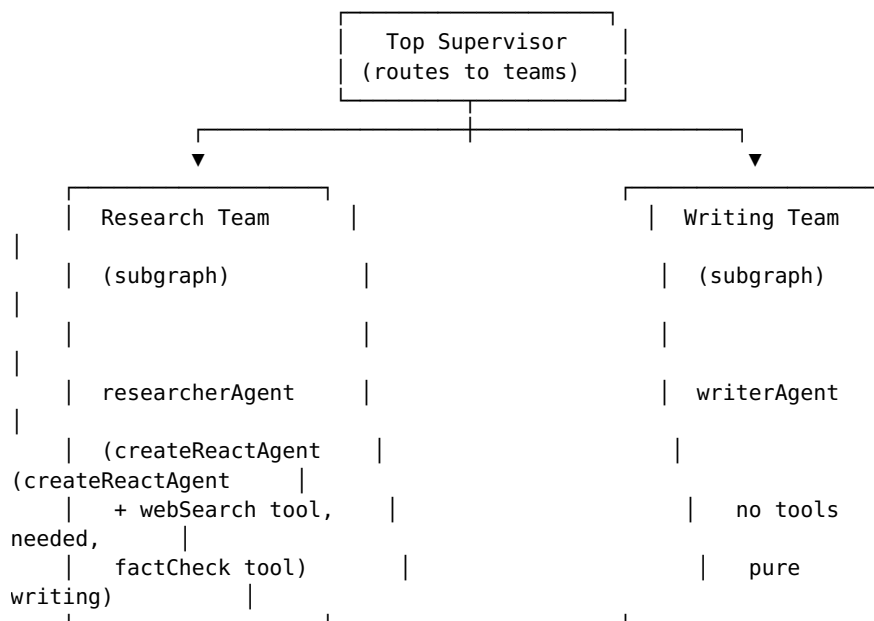
---

## Final Project — Phase 5: A Multi-Team Content Research & Writing System

### What you’re building

A single, end-to-end system that uses **every concept from this phase together**: subgraphs, a hierarchical multi-agent architecture, and tool-calling prebuilt agents. The system takes a topic from the user and produces a short, fact-checked, well-written article about it.

#### Architecture:



This mirrors a real production pattern: a top-level supervisor (Chapter 10) routes between two team subgraphs (Chapter 9), and each team’s “worker” is itself a tool-using prebuilt agent (Chapter 11) rather than a plain LLM call — exactly how you’d realistically combine these three concepts in a real project.

### Step-by-step build

#### Step 1 — Define tools for the Research Team.

Create src/final-project.ts:

```
import "dotenv/config";
import { StateGraph, Annotation, START, END, Command,
```

```

MessagesAnnotation } from "@langchain/langgraph";
import { createReactAgent } from "@langchain/langgraph/prebuilt";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, AIMessage } from "@langchain/core/messages";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0
});

const webSearch = tool(
  async ({ query }: { query: string }) => {
    // Simulated search (swap for a real search API, e.g. Tavily, in
    production).
    const response = await model.invoke(
      `Pretend you searched the web for "${query}". List 3 short,
    plausible facts you "found," each on its own line.`
    );
    return response.content as string;
  },
  {
    name: "webSearch",
    description: "Search the web for facts about a topic.",
    schema: z.object({ query: z.string().describe("What to search
for") }),
  }
);

const factCheck = tool(
  async ({ claim }: { claim: string }) => {
    const response = await model.invoke(
      `Briefly assess whether this claim sounds plausible and well-
    established, or questionable: "${claim}". One sentence.`
    );
    return response.content as string;
  },
  {
    name: "factCheck",
    description: "Check whether a specific factual claim is
    plausible.",
    schema: z.object({ claim: z.string().describe("The specific
    claim to check") }),
  }
);

```

Why these are tools rather than plain subgraph nodes (as we did back in Chapter 9's Lab 9.1): This time we want the *researcher agent itself* to decide, dynamically, how many searches to run and which claims are worth fact-checking — that's a tool-calling decision, not a fixed pipeline. This is a deliberate design choice to combine Chapter 11's pattern with Chapter 9's structure, rather than just repeating Lab 9.1.

## Step 2 — Build the Research Team as a subgraph wrapping a `createReactAgent`.

```

const ResearchTeamState = Annotation.Root({
  ...MessagesAnnotation.spec });

const researcherAgent = createReactAgent({
  llm: model,
  tools: [webSearch, factCheck],
  prompt: "You are a careful researcher. Search for 2-3 facts about
the user's topic, " +
    "fact-check at least one of them, then summarize your
findings in a short paragraph.",
});

```



```
const researchTeam = new StateGraph(ResearchTeamState)
  .addNode("researcher", researcherAgent)
  .addEdge(START, "researcher")
  .addEdge("researcher", END)
  .compile();
```

What's notable here: `researcherAgent` (a compiled `createReactAgent` graph) is itself being used as a node inside `researchTeam` — proving subgraphs aren't limited to nesting only hand-built `StateGraph`s; any compiled graph, including a prebuilt one, works the same way, because `createReactAgent`'s output is also just a `CompiledStateGraph`.

### Step 3 — Build the Writing Team the same way (no tools needed for pure writing).

```
const WritingTeamState = Annotation.Root({
  ...MessagesAnnotation.spec });

const writerAgent = createReactAgent({
  llm: model,
  tools: [], // a tool-using agent with zero tools is valid --
  // it'll just always answer directly
  prompt: "You are a skilled writer. Using the research findings already in the conversation, " +
    "write a short, engaging 3-paragraph article. Do not invent new facts beyond what's provided.",
});

const writingTeam = new StateGraph(WritingTeamState)
  .addNode("writer", writerAgent)
  .addEdge(START, "writer")
  .addEdge("writer", END)
  .compile();
```

### Step 4 — Build the top-level supervisor (same pattern as Lab 10.3).

```
const TopState = Annotation.Root({ ...MessagesAnnotation.spec });

const TopRoute = z.object({ next: z.enum(["researchTeam", "writingTeam", "FINISH"]) });
const topRouter = model.withStructuredOutput(TopRoute);

async function topSupervisorNode(state: typeof TopState.State) {
  const decision = await topRouter.invoke([
    new HumanMessage(
      `Route to "researchTeam" if facts still need gathering, "writingTeam" if research is ready and an article ` +
      `needs writing, or FINISH if a complete article has already been written.\n\n` +
      state.messages.map(m => `${(m as AIMessage).name ?? m.getType()}: ${m.content}`).join("\n")
    ),
  ]);
  return new Command({ goto: decision.next === "FINISH" ? END : decision.next });
}

async function researchTeamNode(state: typeof TopState.State) {
  const teamResult = await researchTeam.invoke({ messages: state.messages });
  return new Command({
    goto: "topSupervisor",
    update: { messages: [new AIMessage({ content: teamResult.messages.at(-1)!.content, name: "researchTeam" })] },
  });
}
```

```

    }

    async function writingTeamNode(state: typeof TopState.State) {
      const teamResult = await writingTeam.invoke({ messages:
state.messages });
      return new Command({
        goto: "topSupervisor",
        update: { messages: [new AIMessage({ content:
teamResult.messages.at(-1)!.content, name: "writingTeam" })] },
      });
    }

    const finalGraph = new StateGraph(TopState)
      .addNode("topSupervisor", topSupervisorNode, { ends:
["researchTeam", "writingTeam", END] })
      .addNode("researchTeam", researchTeamNode, { ends:
["topSupervisor"] })
      .addNode("writingTeam", writingTeamNode, { ends: ["topSupervisor"]
})
      .addEdge(START, "topSupervisor")
      .compile();

```

### Step 5 — Run the full system.

```

const result = await finalGraph.invoke({
  messages: [new HumanMessage("Write me a short article about the
history of tea.")],
});

console.log("\n=== FULL TRACE ===");
for (const m of result.messages) {
  console.log(`\n[${(m as AIMessage).name ??
m.getType()}]:\n${m.content}`);
}

```

**Expected result:** The top supervisor sends the request to researchTeam first. Inside that subgraph, researcherAgent (a full tool-calling ReAct loop) calls webSearch one or more times, calls factCheck on at least one claim, and produces a research summary. Control returns to the top supervisor, which then routes to writingTeam, where writerAgent turns that research into a finished 3-paragraph article. Control returns to the top supervisor one final time, which sees a complete article exists and decides FINISH.

**How to verify the whole system worked correctly:** 1. Re-run with { subgraphs: true } streaming and confirm you can see, in order: the top supervisor's first decision, the researcher agent's internal tool calls (webSearch, factCheck) nested inside the researchTeam step, the writer agent's work nested inside the writingTeam step, and the top supervisor's final FINISH decision. 2. Confirm the final article doesn't contain any claims that weren't present in the research summary — if it does, your writerAgent prompt needs tightening ("Do not invent new facts beyond what's provided" should prevent this, but always verify against real output). 3. Try a thread\_id + checkpointer setup (from Phase 3) on finalGraph.compile({ checkpointer }), and confirm you can use getStateHistory to look back and see each team's contribution as a separate checkpoint in the run.

## What this project proves you can now do

If you've built and verified this system, you've demonstrated, in one working piece of code, that you can: design a state schema that flows correctly across subgraph boundaries (Chapter 9); structure a multi-team hierarchical system with a working top-level router (Chapter 10); and equip individual agents with real tool-calling capability using

both hand-built and prebuilt patterns (Chapter 11). That combination — modular structure, coordinated multi-agent routing, and tool-using specialist agents — is the core shape of the large majority of serious production LangGraph systems you'll encounter or build going forward.

---

*End of Phase 5. Next up in the roadmap: Phase 6 — Production (Functional API, caching, retries, durable execution, observability with LangSmith, and deployment).*